An Optimal Deterministic Adversary for Sorting

Nicholas Nash
ORCID: 0009-0003-0475-7852

8 November 2026

Abstract

This paper describes an optimal *deterministic*, polynomial-time adversary for ordinary sorting. For any correct sorting algorithm, the adversary forces that algorithm to make $(1 - o(1))n \log_2 n$ comparisons. The best previously known adversary forced $\frac{37}{64} n \log_2 n$ comparisons. The adversary retains comparisons in directed acyclic graphs and represents possible orders by a region of real positions satisfying interval bounds and the recorded inequalities. It estimates the fraction of this region's volume retained by each answer through deterministic calculations on bounded graph neighborhoods. Moving items into ordered subproblems controls graph degrees and directed path lengths; a geometric analysis shows that these moves preserve the optimal asymptotic bound.

Methodological disclosure

Except for the abstract, the text of this paper is AI-generated using GPT 6 Astra. The author is working on a clear, human-readable exposition.

1 Introduction

We consider comparison sorting of n distinct labelled keys. A sorter chooses each pair using the answers to its earlier comparisons and must be correct for every strict total order on the labels. A sorting adversary supplies those answers as the comparisons arrive. It is *legal* if every finite sequence of its answers is consistent with at least one total order. The adversary knows n and sees the comparisons, but need not inspect the sorter that generates them. Throughout, log denotes $\log_2$, and ln the natural logarithm; we assume $n \geq 2$.

The classical sorting lower bound guarantees the existence of a hard input for every correct sorter. Here the problem is to produce hard answers by one *deterministic, polynomial-time* procedure. The polynomial bound applies to the adversary's computation; the comparison lower bound applies to the sorter. Without the time restriction, a deterministic optimal adversary is immediate. With randomness, polynomial-time adversaries achieve the optimal asymptotic bound in expectation and with high probability.

Without the time restriction. Keep all total orders consistent with the answers so far. Each comparison partitions them into two classes; answer so as to retain a larger class. After q comparisons, at least $n!/2^q$ orders remain, forcing $q \geq \log(n!)$. This rule is deterministic, but enumerating its classes may require factorial work. A consistent total order is a *linear extension* of the partial order given by the answered comparisons; counting these extensions exactly is #P-complete [8].

Deterministic adversaries from graph entropy. Kahn and Kim [10] already give a deterministic polynomial-time adversary for sorting with partial information. Starting from any given partial order P, it answers comparisons online and forces $\Omega(\log e(P))$ comparisons, where $e(P)$ is the number of linear extensions of P. Their method uses graph entropy and convex optimization via the ellipsoid method. For ordinary sorting, the initial partial order has no relations, so $e(P) = n!$ and their guarantee is $\Omega(n \log n)$. Thus deterministic polynomial-time adversaries are available even for arbitrary initial partial information. The remaining question for ordinary sorting is whether such an adversary can force $(1 - o(1))n \log n$ comparisons.

With randomness. An efficient adversary can simply choose a uniformly random permutation of the labels, keep it hidden, and answer from the ranks in that permutation. For every fixed correct sorter, the usual comparison-tree argument gives an expected cost of at least $\log(n!)$. It also gives a high-probability bound. At most 2^q permutations can terminate at leaves of depth at most an integer q, so the comparison count C satisfies

$$\mathbb{P}(C \le q) \le \frac{2^q}{n!}.$$

Thus $C \ge \log(n!) - O(\log n)$ with probability $1 - o(1)$. Generating an exactly uniform permutation takes expected polynomial bit time; subsequent answers require only rank lookups. A fixed polynomial time bound is obtained by using an almost uniform permutation, with a vanishing additional failure probability. These guarantees also hold for an always-correct randomized sorter whose random bits are independent of the adversary's: condition on the sorter's random choices and apply the same argument.

For any prescribed permutation, a correct sorter can first check its $n - 1$ consecutive pairs, finish if all checks agree, and otherwise use a general sorting algorithm. That permutation then costs only $n - 1$ comparisons. The deterministic adversary must choose its answers in response to the questions it receives.

Randomized approximation of the number of linear extensions. Randomization also makes it possible to estimate how many linear extensions remain after each possible answer. Given the current partial order P, assign each label a real coordinate in $(0, 1)$ and form the region

$$K(P) = \{X \in (0,1)^n : X_i < X_j \text{ whenever } i <_P j\}.$$

Each allowed total order occupies volume $1/n!$ in the unit cube. Hence $\mathrm{vol}(K(P)) = e(P)/n!$ [7]. The two possible answers to a comparison cut this region into two parts, and their volumes are proportional to their counts of allowed orders.

Dyer, Frieze, and Kannan [9] give a randomized polynomial-time algorithm for relative volume approximation and explicitly note its application to linear extensions. To use it here, first check whether the queried relation is already forced by P. If so, return it directly. Otherwise both directions are feasible and give positive-volume regions. Estimate their volumes and choose the larger estimate. When both estimates have relative error at most ε, the chosen part retains at least $(1 - \varepsilon)/2$ of the current volume. The counting argument then forces

$$q \ge \frac{\log(n!)}{1 - \log(1 - \varepsilon)} = (1 - O(\varepsilon)) \log(n!)$$

comparisons. Taking $\varepsilon = 1/\lceil \log(n + 1) \rceil$ gives $n \log n - O(n)$.

For a high-probability guarantee, allow $Q = 2n \lceil \log(n + 1) \rceil$ queries before fixing a consistent order. Amplify each volume estimate to failure probability at most n^{-4}. Each call uses fresh randomness, so this bound holds conditional on the preceding history. A union bound over at most $2Q$ estimates gives total failure probability $O(\log n / n^3)$, even for adaptive queries. These

accuracy requirements preserve polynomial time. Every run remains legal, including when an estimate fails, because only feasible answers are offered.

The issue is therefore *derandomization*: obtain the optimal asymptotic comparison bound with a single deterministic response rule whose work per comparison is polynomial in n. Kaplan, Zamir, and Zwick [5, Section 7] explicitly ask for an adversary forcing $(1 - o(1))n \log n$ comparisons. Our main theorem answers this question.

Theorem 1.1. *There is a deterministic legal adversary, initialized using only n, that performs a fixed polynomial number of bit operations per comparison and forces every correct comparison sorter to make*

$$(1 - o(1))n \log n$$

comparisons.

Merge sort uses at most $n \lceil \log n \rceil$ comparisons, so this bound is asymptotically optimal. The running-time guarantee is asymptotic and involves very large parameter thresholds.

Section 2 reviews the earlier adversaries. Section 3 gives our data structure and comparison procedure. Items are stored at nodes of a rooted tree, with a directed acyclic graph recording comparisons within each node. Interval bounds and the graph's inequalities define a region of real positions representing the possible orders there. The adversary computes lower bounds on the fraction of this region's volume retained by each answer. Three technical ingredients make this possible.

Certifying comparisons with enclosing intervals. After propagating interval bounds through the full graph, we perform finite computations on a bounded neighborhood. Each computation returns intervals enclosing uniformly distributed feasible positions. Disjoint intervals establish a comparison, so counting these cases gives certified lower bounds on the probabilities of the two answers. Degree and path-length bounds ensure that almost all returned intervals are short; an unsuccessful certificate then implies that the queried positions are confined to very short intervals. This identifies an item to refine (Section 6).

Choosing positions with large remaining volume. Moving an item into a child fixes its position in the parent. We choose a binary fraction with the fewest digits in a suitable interval. We first try to certify substantial probability inside an interval and on both sides. Log-concavity then gives every point inside a density comparable to the peak. If certification fails, the middle half of the item's effective interval supplies the required volume gain (Section 5.3).

Bounding the cost of graph simplification. Section 4 compares the volume gained by moving items with the probability of choosing their children in a counting experiment. Failed comparison certificates and long directed paths supply enough volume gain to pay for these choices. Moves triggered by high degree are charged to the comparisons whose arcs they remove. The experiment gives a lower bound on the number of orders represented by any state, connecting the analysis to the sorter's task of determining one order.

Finally, Section 7 lets the degree and path-length bounds grow slowly with n, keeping the computation polynomial as the comparison coefficient approaches one. After $n \lceil \log n \rceil$ answers, the adversary can choose a fixed order consistent with its answers and use that order thereafter. This bounds the lifetime of the recursive construction.

Appendix A proves the density estimates, Appendix B proves the local enclosure theorem, and Appendix C collects the parameters. The manuscript source is available online.[1]

[1] https://github.com/nicknash/sorting-adversary-proofs/tree/main/for-humans

2 Related Work

2.1 Binary-tree adversaries

Atallah and Kosaraju [1] gave an efficient sorting adversary with a lower bound of $\frac{1}{2}n \log n$ comparisons. Related adversaries occur in the work of Borodin, Guibas, Lynch, and Yao [2] on searching after partial sorting, Richards and Vaidya [3] on the distribution of comparisons among keys, and Brodal, Chaudhuri, and Radhakrishnan [4] on maintaining the minimum. The binary-tree rule of Borodin et al. and Brodal et al. provides the starting point for our construction.

Place every item at the root of an ordered binary tree. Items in disjoint subtrees have the left-to-right order of those subtrees. A comparison changes the placement as follows.

- If both items occupy the same node, move one to its left child and the other to its right child, and answer accordingly.
- If one item is at an ancestor of the other, move the higher item to the child whose subtree does not contain the lower item.
- If their nodes are unrelated by ancestry, answer from their left-to-right order, without moving either item.

For a concrete example, start with A, B, C, D at the root. Answering $A < B$ moves A to the left child and B to the right, while C, D remain at the root. If the sorter next compares A and C, the ancestor rule moves C to the right child and gives $A < C$. Comparing D and B then moves D to the left child and gives $D < B$. The two children now contain $\{A, D\}$ and $\{B, C\}$. A subsequent comparison of D with C is answered $D < C$ without moving either item. The orders within $\{A, D\}$ and within $\{B, C\}$ remain unresolved. Figure 1 shows the placements.

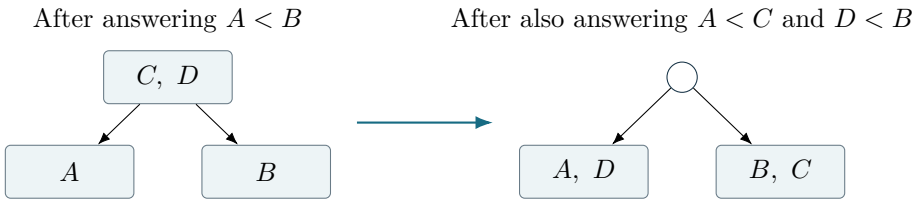


Figure 1: The binary-tree adversary. The first comparison separates two items at the same node. The next two comparisons each move an item from an ancestor into the opposite child. In the final state, comparing D with C requires no movement.

Each comparison increases the sum of item depths by at most two. At completion, occupied nodes must be distinct and unrelated by ancestry: otherwise two represented items can occur in either relative order, leaving an unresolved comparison. Associate the root with $(0, 1)$, and associate each child with the corresponding half of its parent's interval. A node of depth d_i therefore represents an interval of length 2^{-d_i}. At completion these intervals are disjoint, giving

$$\sum_i 2^{-d_i} \le 1, \qquad 2^{-\frac{1}{n}\sum_i d_i} \le \frac{1}{n}\sum_i 2^{-d_i} \le \frac{1}{n}.$$

Convexity gives the bound on $2^{-\frac{1}{n}\sum_i d_i}$. Hence $\sum_i d_i \ge n \log n$, and at least $\frac{1}{2}n \log n$ comparisons are necessary [4, Section 4.1].

The tree representation can settle relations that have never been queried. For example, start again with four items at the root and answer $A < B$ and $C < D$. Placing A, C in the left child and B, D in the right child also commits the adversary to $A < D$ and $C < B$. Neither relation follows from those two answers alone.

2.2 The adversary of Kaplan, Zamir, and Zwick

Kaplan, Zamir, and Zwick [6] replace each step of a four-child tree by a small directed acyclic graph (DAG), allowing an item to move towards a group of consecutive children before selecting one child. Number the children $0, 1, 2, 3$. The intermediate node $[i, j]$ represents the possible destinations $i, i+1, \ldots, j$. The parent itself is $[0, 3]$, and $[i, i]$ is child i. For $i < j$, a downward step removes one endpoint of the range, reaching $[i, j-1]$ or $[i+1, j]$. Figure 2 shows this interval lattice. For example, $[0, 2]$ and $[1, 3]$ both lead to $[1, 2]$; the intermediate nodes have shared descendants. The same construction is repeated at every node of the four-child tree.

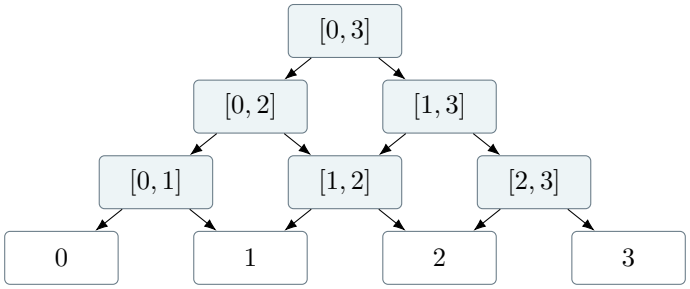


Figure 2: One step of the four-child tree in the adversary of Kaplan, Zamir, and Zwick [6, Section 5]. An intermediate node records a range of possible children. In particular, an item at $[1, 3]$ has not committed to child 1, 2, or 3.

Within a node, the adversary stores unpaired items and disjoint ordered pairs. There can be many items at a node, but its retained comparison components have at most two vertices. Tree nodes and intermediate nodes representing three children may store pairs; intermediate nodes representing two children store only unpaired items. Comparing two unpaired items A, B at a tree node can therefore record $A < B$ as a pair without moving either item.

A comparison between pairs in different nodes illustrates the role of the lattice. Suppose $A < B$ is retained at $[0, 2]$ and $C < D$ at $[1, 3]$. On comparing A and C, one permitted response is $A < C$: move A into child 0 and B into $[1, 2]$, leaving the pair $C < D$ at $[1, 3]$. Figure 3 shows the change. The old answer $A < B$ is now represented by the items' positions, while $C < D$ remains a retained pair [6, Table 11].

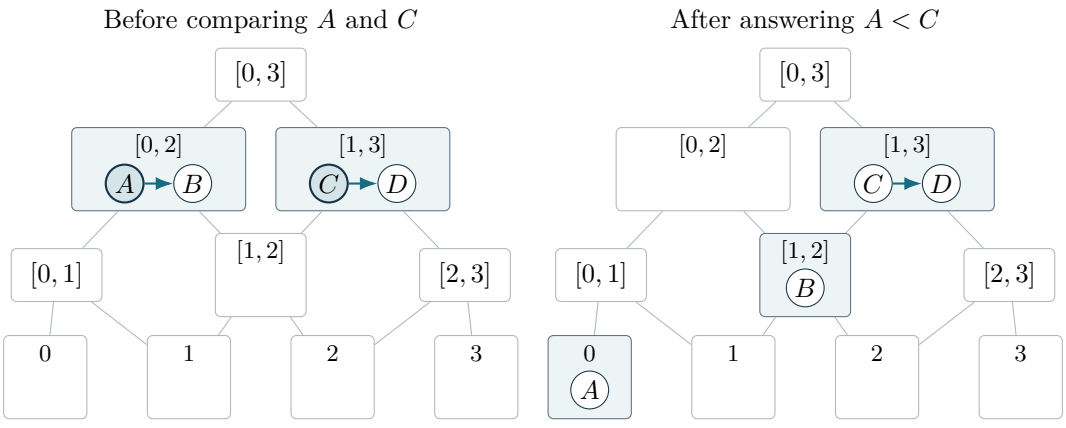


Figure 3: A comparison between pairs at different nodes in the adversary of Kaplan, Zamir, and Zwick [6, Table 11]. Shaded items are queried. Blue arcs are retained comparisons; gray lines are lattice edges. The new answer $A < C$ is enforced by the disjoint child ranges assigned to A and C.

A retained pair also restricts its members' possible destinations: they cannot enter the same child of the underlying four-child tree. For example, if $A < B$ is stored at $[0, 2]$, then A can enter only child 0 or 1, and B can enter only child 1 or 2. The pair's order thus narrows

each member's possible destinations, and the adversary uses these restrictions when choosing subsequent comparison answers [6, Section 5].

Their analysis measures progress using item depths, intermediate placements, and retained pairs. The comparison rules bound the increase in this measure and give a lower bound of $\frac{37}{64}n \log n$ comparisons, improving the earlier $\frac{1}{2}n \log n$ bound. Their concluding discussion proposes allowing more complicated comparison components than pairs, and asks whether an efficient deterministic adversary can force $(1 - o(1))n \log n$ comparisons [6, Section 6].

3 The adversary

We describe the data structure maintained by our adversary and how it is updated in response to comparison queries from the sorting algorithm.

Kaplan, Zamir, and Zwick [6] retain disjoint ordered pairs within a node. Our adversary maintains a rooted tree with a DAG at each node, so several retained comparisons can involve the same item. For example, it can keep $a < b$ and $a < c$ together while leaving the order of b and c undecided. This lets it represent richer partial orders before moving items further down the tree.

The computational challenge is to use these richer orders without having to approximately count the linear extensions of an entire DAG. We periodically simplify a node's DAG by removing vertices and moving their items into child nodes, creating children when needed. Constraints at the parent preserve the comparisons involving the moved items, while their order within a child can remain unresolved. These simplifications allow comparison answers to be chosen by deterministic calculations on small neighborhoods in that node's DAG. The analysis must show that the resulting loss of possible orders is small enough to retain the optimal comparison bound.

3.1 The data structure

Our data structure is a rooted tree. Each node stores item records and a single directed acyclic graph (DAG), whose vertices are the items recorded at that node. We call this its *in-node DAG*. Figure 4 shows an example.

Each item has a record (x, X_x, a_x, b_x) stored at exactly one node. The label x identifies its key, and X_x is a symbolic variable for its position among the items at that node. The fields a_x, b_x restrict this variable to the open interval $(a_x, b_x) \subseteq (0, 1)$. The bounds a_x, b_x are fractions whose denominators are powers of two; we call such numbers *dyadic rationals*. We call an item *free* at its node while its position remains variable.

In an in-node DAG, an arc $x \to y$ records a comparison answered $x < y$ between two items stored there, imposing $X_x < X_y$ on their positions. Each answered comparison between free items contributes at most one arc.

A *feasible assignment* at a node is a choice of real values for its position variables satisfying $a_x < X_x < b_x$ for every item x and $X_x < X_y$ for every arc $x \to y$ in its in-node DAG.

Initially the tree consists of just the root, containing all records with bounds $a_x = 0$ and $b_x = 1$. The position variables X_x are unassigned, and the root's in-node DAG has no arcs. Thus every assignment in $(0, 1)^n$ is feasible, and every total order can be obtained by assigning distinct positions.

Each child is attached to a fixed position $c \in (0, 1)$ in its parent, called its *anchor*. A node has at most one child at each anchor. For any feasible assignment at the parent, an item with position below c precedes every item in that child's subtree; an item with position above c follows every item in it. Subtrees of different children are ordered by their anchors. All items of each child subtree therefore occur consecutively in the resulting total order. Within the child, position

variables again range over $(0, 1)$, and its in-node DAG and interval bounds describe the possible orders inside this block.

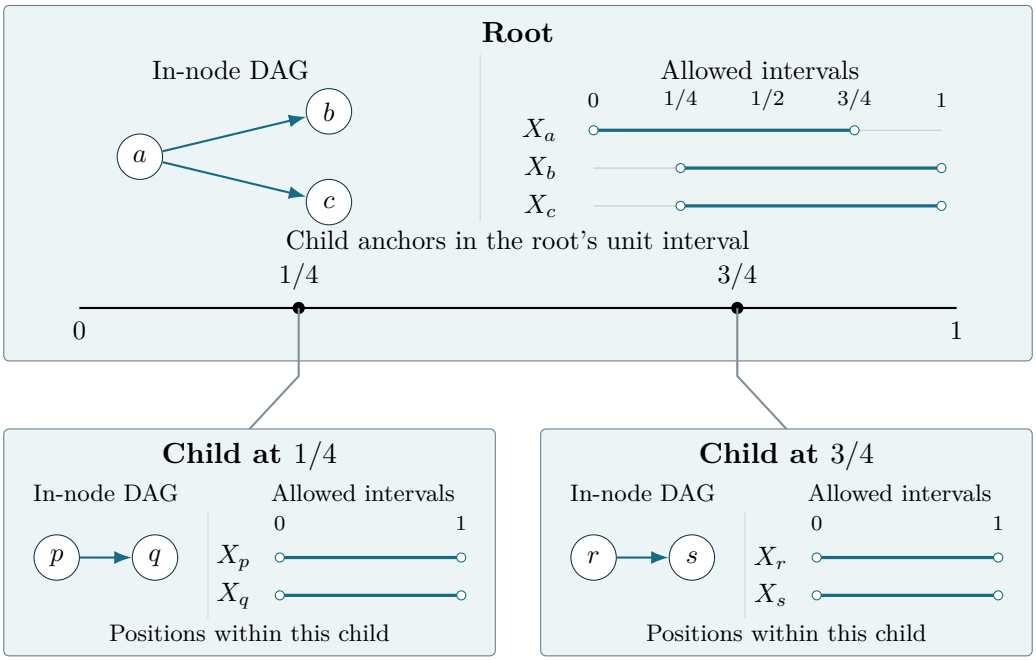


Figure 4: A state of the adversary. Each large box is a tree node, containing an in-node DAG and one open interval for each of its items.

In the root of Figure 4, the in-node DAG records $a < b$ and $a < c$. To read an order from this state, choose feasible positions at each node, avoiding ties with other items or child anchors. List the items and anchors from left to right, and recursively replace each anchor by its child's order. For example, root positions $(X_a, X_b, X_c) = (1/3, 1/2, 7/8)$ give

$$p < q < a < b < r < s < c.$$

Other feasible choices give other orders. Every order represented this way must respect all previous answers to the sorter. We call the sequence of these answers the *transcript*.

3.2 Answering a comparison

Algorithm 1 shows the algorithm the adversary uses to answer a comparison by the sorter.

To compare A and B, start at the root and descend into a child whenever its subtree contains both items. Stop when at least one item is stored at the current node, or when the items lie in different child subtrees. At this node, use an item's free coordinate if it is stored here, and the corresponding child anchor otherwise. In Figure 4, comparing p with q descends into the left child; comparing p with r uses the anchors $1/4$ and $3/4$; and comparing a with p compares X_a with $1/4$ at the root.

Two distinct anchors determine the answer immediately. The other cases use bounds implied by the in-node DAG. If there is an arc $y \to x$, every lower bound on X_y is also a lower bound on X_x; upper bounds propagate in the reverse direction. Write $y \preceq x$ when $y = x$ or a directed path leads from y to x. The resulting *effective interval* is

$$\ell_x = \max_{y \preceq x} a_y, \qquad r_x = \min_{y \succeq x} b_y, \qquad W_x = r_x - \ell_x. \tag{1}$$

These endpoints can be computed in two passes through the in-node DAG: lower endpoints in topological order, then upper endpoints in reverse order. We compute them when needed, keeping the record's bounds a_x, b_x as introduced by answers and refinements.

For a query involving a free coordinate, consider the uniform distribution on feasible positions at this node. The subroutine `Certify` computes two nonnegative rational numbers p_{AB}, p_{BA} that are lower bounds on the probabilities of $A < B$ and $B < A$. Fix a rational tolerance $0 < \eta < 1/32$. If the larger certified fraction is at least $1/2 - \eta$, choose that answer, breaking a tie in favor of $A < B$. Otherwise, move a queried free item into a child and restart the query. We call this move a *refinement*, denoted `Refine`(x); Algorithm 2 in Section 3.3 gives the operation. Section 4 shows why a failed certificate makes such a refinement affordable. For two free items, the events are $X_A < X_B$ and $X_B < X_A$; for a free item and an anchor, they compare its coordinate with the fixed anchor. An answer between two free items adds an arc to the in-node DAG; an answer involving an anchor adds an interval bound.

We also control the in-node DAGs using integers $k, H \geq 2$. During the recursive construction, each vertex has at most k neighbors, counting both incoming and outgoing arcs, and each directed path has fewer than $2H$ vertices. Before calling `Certify`, we first remove any path of H vertices by refining its entire list of items. If a queried free item has degree k, we refine that item too. Each of these steps restarts the query. When `Certify` is reached, the queried vertices have degree below k and the current node has no H-vertex path. Adding one arc therefore preserves the bounds needed throughout the interaction: degrees rise by at most one, and a new path joins two old paths.

The following lemma supplies the guarantee needed by the comparison rule. Section 6 implements `Certify` by counting finite calculations in which enclosing intervals establish one of the two answers.

Lemma 3.1 (Certified comparison probabilities)**.** *Suppose a node admits a feasible assignment, and its in-node DAG has degree at most k and directed paths of at most $2H$ vertices. For two distinct free coordinates, or a free coordinate and a dyadic threshold,* `Certify` *deterministically returns nonnegative rational lower bounds on the probabilities of the two answers. If one queried free coordinate has effective interval $(0, 1)$, their maximum is at least $1/2 - \eta$.*

The recursive construction is needed for at most

$$Q(n) = n \lceil \log n \rceil \tag{2}$$

answers. Record these answers in a list $\mathcal{T}$, initially empty. If another query arrives after $Q(n)$ answers, choose a fixed total order consistent with $\mathcal{T}$ and use it for every subsequent answer. Section 7 constructs this order by a deterministic topological sort. A sorter reaching this point has already made at least $n \log n$ comparisons.

Algorithm 1: `Compare`(A, B): answer a comparison query.

Input: Distinct item labels A, B and the current adversary state.
Output: An answer $A < B$ or $B < A$, with the state updated.
if *fixed ranks have been chosen* **then**
  return their order of A, B
if $|\mathcal{T}| = Q(n)$ **then**
  Form the DAG of answers in $\mathcal{T}$ and choose a topological order, breaking ties by label
  Store its ranks and **return** their order of A, B
while *true* **do**
  $v \leftarrow$ the root
  while *A and B lie in the same child subtree of v* **do**
    $v \leftarrow$ that child
  if *A and B lie in different child subtrees of v* **then**
    answer $\leftarrow$ the left-to-right order of their child anchors
    break
  if *the in-node DAG at v has a directed path on H vertices* **then**
    $L \leftarrow$ the list of labels on the selected path
    foreach $x \in L$ *in label order* **do**
      `Refine`(x)
    continue
  if *a queried free item has degree k in the in-node DAG at v* **then**
    `Refine`(the selected such item)
    continue
  $(p_{AB}, p_{BA}) \leftarrow$ `Certify`(v, A, B, η)
  if $\max(p_{AB}, p_{BA}) \geq 1/2 - \eta$ **then**
    answer $\leftarrow A < B$ if $p_{AB} \geq p_{BA}$, and $B < A$ otherwise
    Store *answer* as an arc in the in-node DAG at v or as an interval bound
    break
  `Refine`(the selected queried free item)
Append *answer* to $\mathcal{T}$
return *answer*

Every restart refines at least one item. Its new coordinate in the child is isolated and has interval $(0, 1)$. Further refinements cannot give that coordinate an incident arc or narrow its interval. Lemma 3.1 then guarantees that a query involving it is certified, so it cannot be selected again during this call. Thus a call makes at most n refinements; Section 5.4 gives the full proof.

For deterministic selection, choose the smallest eligible item label and the lexicographically first eligible directed path. Complete each selected path's list even if earlier refinements break the path.

3.3 Moving an item into a child

Algorithm 2 gives `Refine`(x), which moves x from its current node into a child. It computes x's effective interval and calls `ChooseAnchor` to select a position c inside it.

We first try to find an interval with substantial probability inside and on both sides. If that calculation fails, we use the middle half of the item's effective interval.

Lemma 3.2 (A central interval)**.** *Under the hypotheses of Lemma 3.1,* `CentralInterval` *deterministically returns either failure or dyadic* $u < v$ *in* (ℓ_x, r_x) *satisfying*

$$\mathbb{P}(X_x \le u) \ge \frac{1}{4}, \qquad \mathbb{P}(u \le X_x \le v) \ge \frac{1}{4}, \qquad \mathbb{P}(X_x \ge v) \ge \frac{1}{4}.$$

Section 6 implements this routine using the same enclosing intervals as `Certify`.

We select anchor positions using a fixed infinite binary tree, the *implicit anchor tree*, shown in Figure 5. The same tree is used at every node of the adversary's tree. Each anchor-tree node u has an interval $I(u) = [a, b]$ and a position $c(u) = (a + b)/2$; write $|I(u)| = b - a$. The root has interval $[0, 1]$; the left and right children of u have intervals $[a, c(u)]$ and $[c(u), b]$, respectively.

At depth m, the 2^m nodes have intervals of length 2^{-m} and positions

$$c = (i + \tfrac{1}{2})2^{-m}, \qquad i = 0, \dots, 2^m - 1. \tag{3}$$

We call m the *anchor depth*, always measured in the anchor tree. These positions are dyadic rationals, so refinement keeps the interval endpoints a_x, b_x dyadic. A finite path from the anchor tree's root specifies an anchor. Algorithm 3 in Section 5.3 gives the rule for choosing a central interval and selecting its shallowest anchor. Equivalently, this is the binary fraction in that interval with the fewest fractional digits.

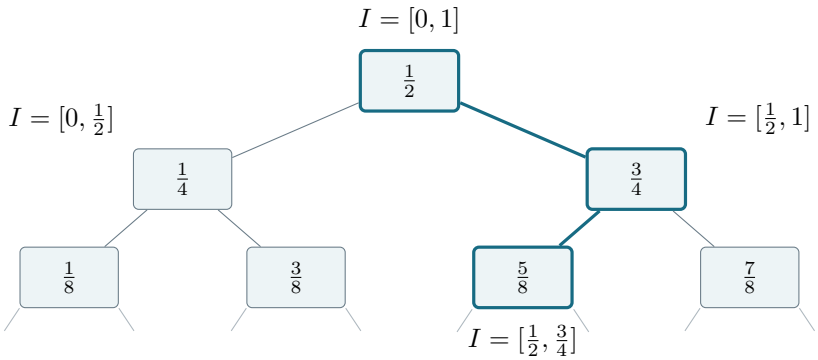


Figure 5: The implicit anchor tree. The highlighted path selects 5/8 at depth 2.

Fixing $X_x = c$ places x in the child block anchored at c. The arcs incident to x in that in-node DAG become interval bounds: $y \to x$ gives $X_y < c$, and $x \to z$ gives $X_z > c$. Remove x and these arcs from the parent, and retain the new bounds on its neighbors. Insert x into the child anchored at c, creating it if needed. In that child, x receives a fresh position variable with interval $(0, 1)$ and no incident arcs; the child's existing constraints remain unchanged. Figure 6 illustrates reuse of an existing child.

Algorithm 2: `Refine`(x): move an item into a child.

Input: A free item x and the current adversary state.
Output: The updated state, with x stored in a child of its former node.
$v \leftarrow$ the node storing x
Compute x's effective interval using (1)
$c \leftarrow$ `ChooseAnchor`(x)
foreach *arc* $y \to x$ *in the in-node DAG at* v **do**
  $b_y \leftarrow \min(b_y, c)$
foreach *arc* $x \to z$ *in the in-node DAG at* v **do**
  $a_z \leftarrow \max(a_z, c)$
Remove x's record from v and its vertex and incident arcs from the in-node DAG at v
if v *has no child anchored at* c **then**
  Create a child of v at c, with no records and an empty in-node DAG
$u \leftarrow$ the child of v anchored at c
Insert a record $(x, X_x, 0, 1)$ at u, with a fresh, unassigned position variable X_x
Insert x as an isolated vertex in the in-node DAG at u

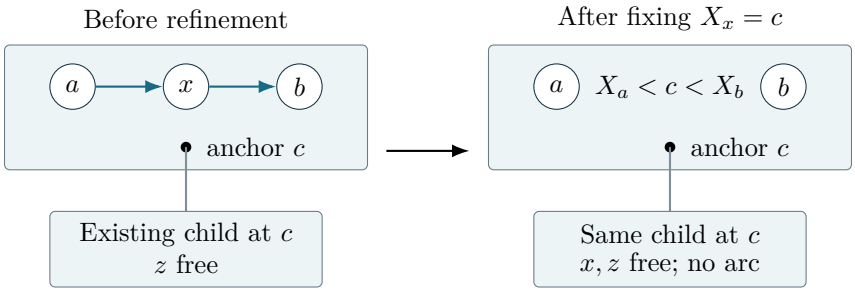


Figure 6: The two arc inequalities become bounds on the remaining parent coordinates. The new coordinate of x orders it within the child, where both $x < z$ and $z < x$ remain possible. The anchor c locates the child block among the parent items; the child's constraints determine the orders within that block.

Thus the parent fixes where the child block occurs, while the order of x within that block stays open. Lemma 5.2 proves that this operation preserves all answered comparisons.

Refinement can impose additional order relations. For example, suppose the root contains A, B, C with only $A < B$ recorded. Refining A at $1/2$ restricts B to $(1/2, 1)$. Refining C into the same child then leaves both $A < C < B$ and $C < A < B$ possible, while fixing the additional relation $C < B$. Section 4 shows that we can afford this loss of possible orders while retaining the optimal asymptotic comparison bound.

Table 1 collects the thresholds used by Algorithms 1 and 2 above. In the next section, we explain why the adversary forces any correct sorter to perform many comparisons.

4 Why the adversary forces many comparisons

Recall from Algorithm 1 that a larger degree threshold k lets the adversary retain more comparisons involving each item before moving it into a child. Increasing k lets the adversary force a comparison count closer to $n \log n$. The following bound quantifies this dependence.

Theorem 4.1 (Comparison bound)**.** *For every sufficiently large integer k, the adversary of Algorithm 1, with the remaining parameter choices in Table 1, is deterministic and legal, and every call terminates. Every correct sorter makes at least*

$$\frac{\log(n!) - n}{1 + O(\log k/k)} \tag{4}$$

comparisons.

For this section, fix a sufficiently large integer k. Section 7 chooses k as a function of n so that it grows without bound while the adversary's running time remains polynomial.

We will bound the number of total orders represented by any state. The bound combines the freedom within nodes with weights assigned to child choices. A single inequality relating refinement cost to volume change will then explain how each of the three refinement rules preserves enough orders.

4.1 Volume changes

We first measure the freedom left within the individual nodes. At a node v, the feasible assignments form the region

$$K_v = \{X : a_x < X_x < b_x \text{ for every free } x, \quad X_x < X_y \text{ for every in-node arc } x \to y\}. \tag{5}$$

Write $V_v = \mathrm{vol}(K_v)$ and take the product over the adversary's tree:

$$Z = \prod_v V_v. \tag{6}$$

Each item has a position variable at exactly one node, so the product region has dimension n. A node with no free items contributes volume one. Initially $Z = 1$; whenever every node has positive volume, $0 < Z \leq 1$.

Choose the certificate tolerance to be

$$\eta = \frac{1}{100k}. \tag{7}$$

By Lemma 3.1 and the acceptance test in Algorithm 1, each chosen answer retains at least $1/2 - \eta$ of its node's volume. Its decrease in $\log Z$ is therefore at most

$$a_\eta = -\log(1/2 - \eta) = 1 + O(1/k). \tag{8}$$

A comparison between two child anchors leaves Z unchanged.

Refinement fixes an item's position in its parent and gives it a fresh unit coordinate in a child. Write

$$g = \log \frac{Z_{\text{after}}}{Z_{\text{before}}} \tag{9}$$

for the resulting change in $\log Z$. We call g the refinement's *volume gain*; it can be negative.

The algorithm must preserve positive node volumes and the bounds needed by the probability routines. It must also return answers consistent with the earlier comparisons. The following proposition gives these guarantees.

Proposition 4.2 (Consistency and termination)**.** *During the recursive phase, every reached state has positive node volumes and represents only orders consistent with the transcript. Every call to Algorithm 1 terminates and performs at most n refinements. Every in-node DAG has degree at most k and directed paths of fewer than $2H$ vertices throughout this phase.*

Section 5.4 proves Proposition 4.2.

4.2 Counting the orders represented by a state

The product Z measures freedom within individual nodes. For example, two items stored alone in separate children can each retain the full interval $(0, 1)$, giving $Z = 1$, even though the child anchors fix their order. To account for this restriction, assign a positive weight $w(c)$ to each child at anchor c, with the weights at each parent summing to at most $1/2$.

For each item, multiply the child weights along the path from the root to its storage node. Let $\mathcal{W}$ be their product over all items. An item stored at the root contributes one. Thus each edge weight appears once for every item in its subtree.

Lemma 4.3 (Number of represented orders)**.** *If the state represents N_{ord} total orders, then*

$$N_{\mathrm{ord}} \geq n!\, 2^{-n} Z \mathcal{W}. \tag{10}$$

Proof. On the fixed state tree, start at the root and enter each existing child c with probability $w(c)$. With the remaining probability, stop and choose a uniform coordinate in $(0, 1)$ at the current node. The stopping probability is at least $1/2$, and the finite tree ensures termination.

Draw one such position independently for every label. The state tree orders these positions by comparing free coordinates and child anchors, then ordering positions within a child recursively. Ties have probability zero. All labels have the same distribution, so each of the $n!$ relative orders has probability $1/n!$.

Consider the event that every label stops at its actual storage node and the coordinates satisfy all in-node constraints. The required paths have probability $\mathcal{W}$, and stopping at their endpoints contributes at least 2^{-n}. Conditional on these choices, the coordinates are independent and uniform, so the probability of satisfying the constraints is Z. Every order produced by this event is represented by the state. Its probability is therefore at most $N_{\mathrm{ord}}/n!$, proving the lemma. □

Define the cost of choosing a child by

$$\ell(c) = -\log w(c). \tag{11}$$

Records only move downward. Their current paths therefore contain exactly the edge traversals made during refinement, counted once for each moved item. Consequently

$$-\log \mathcal{W} = \sum_{\text{refinements } r} \ell(c_r). \tag{12}$$

4.3 One bound for the cost of refinement

We now choose the child weights so that their costs can be compared with the volume changes caused by refinement. For an anchor c at depth m in the implicit anchor tree, set

$$w(c) = \frac{1}{2k} \left(\frac{1 - 1/k}{2} \right)^m. \tag{13}$$

There are 2^m anchors at depth m, so their total weight is

$$\sum_c w(c) = \frac{1}{2k} \sum_{m \geq 0} (1 - 1/k)^m = \frac{1}{2}.$$

In particular, the existing children at any node satisfy the weight condition of Lemma 4.3. The cost is

$$\ell(c) = \gamma m + \log k + 1, \qquad \gamma = 1 - \log(1 - 1/k). \tag{14}$$

The anchor routine will provide the following guarantee. It says that choosing a child costs approximately the volume gained by fixing the parent coordinate, with an additive term of order $\log k$.

Lemma 4.4 (Cost and volume change of a refinement)**.** *There are coefficients $a = 1 + O(1/k)$ and $b = O(\log k)$, with $a \geq 1$ and $1 \leq b < k + 1$, such that every refinement satisfies*

$$g \geq -4, \qquad \ell(c) \leq ag + b. \tag{15}$$

A refinement made after unsuccessful comparison certification has gain $g > k$.

Section 5.6 proves the lemma. The additive term can be absorbed when a refinement has a large gain, or when a batch has a large total child cost. The path rule supplies the latter. Choose

$$H = 2^k. \tag{16}$$

Lemma 4.5 (Distinct children for a path)**.** *The labels of a complete path batch enter distinct children of their parent in the adversary's tree.*

Section 5.5 proves this property using the order already determined along the selected path.

4.4 The comparison bound

Proof of Theorem 4.1. Proposition 4.2 gives consistency and termination during the recursive phase. At the cutoff, at least one total order respects all recorded answers, so their directed graph is acyclic. Its chosen topological order extends these answers and makes every later answer consistent with them. All procedures and tie rules are deterministic.

If the sorter reaches the cutoff $Q(n)$ in (2), it has already made at least $n \log n$ comparisons, which suffices for the claimed bound. Otherwise it finishes after $q < Q(n)$ calls to the recursive procedure. We show how each refinement rule accounts for the additive term in (15).

Unsuccessful comparison certification. The resulting refinement has $g > k$, so

$$\ell(c) \leq ag + b \leq (a + b/k)g.$$

Complete path batches. Write $L = \sum_i \ell(c_i)$ and $G = \sum_i g_i$ for a batch of H moves. Its children are distinct by Lemma 4.5. Their weights sum to at most $1/2$, so the arithmetic–geometric mean inequality gives

$$\prod_{i=1}^{H} w(c_i) \leq \left(\frac{1}{2H}\right)^H, \qquad L \geq H \log(2H) = H(k+1).$$

Summing (15) and substituting $H \leq L/(k+1)$ gives

$$L \leq aG + bH \leq aG + \frac{b}{k+1} L.$$

Thus $L \leq \lambda G$, where

$$\lambda = \frac{a}{1 - b/(k+1)} = 1 + O(\log k/k). \tag{17}$$

The assumptions $a, b \geq 1$ give

$$\lambda - a = \frac{ab}{k + 1 - b} \geq \frac{b}{k},$$

so the same factor λ also covers refinements made after unsuccessful comparison certification.

Degree refinements. The degree rule refines a queried vertex of degree k. The bound $g \geq -4$ gives

$$\ell(c) \leq \lambda g + \sigma, \qquad \sigma = b + 4(\lambda - a) = O(\log k). \tag{18}$$

After q comparisons, there have been at most q/k degree refinements: each deletes k in-node arcs, each comparison inserts at most one arc, and an inserted arc can be deleted only once. Hence the total refinement cost satisfies

$$\sum_r \ell(c_r) \leq \lambda \sum_r g_r + \frac{\sigma}{k} q. \tag{19}$$

Each comparison decreases $\log Z$ by at most a_η, while a refinement changes it by g_r. Since initially $Z = 1$,

$$\sum_r g_r \le \log Z + a_\eta q.$$

Combining this with the remaining-order bound (10) and (12) gives

$$\log N_{\text{ord}} \ge \log(n!) - n + (1-\lambda)\log Z - \left(\lambda a_\eta + \frac{\sigma}{k}\right) q. \tag{20}$$

Here $(1-\lambda)\log Z \ge 0$, and

$$\lambda a_\eta + \frac{\sigma}{k} = 1 + O(\log k/k).$$

Thus many orders remain until nearly $\log(n!)$ comparisons have been made. At termination $N_{\text{ord}} = 1$, giving

$$q \ge \frac{\log(n!) - n}{1 + O(\log k/k)}.$$

This proves (4). □

The next section proves the refinement and consistency guarantees used above.

5 Refinement and its guarantees

We now prove the refinement guarantees used in Section 4: moving an item preserves the answers already given, and the resulting change in volume controls the cost of choosing a child.

5.1 Preserving the represented orders

We use the interpretation of the state given in Section 3. Assignments with equal free coordinates, or with a free coordinate equal to an anchor, lie on a finite union of sets of volume zero and are excluded when reading an order. We maintain the following invariant:

> Every represented total order respects every comparison already answered.

The state may discard some orders that still respect the answers. Legality requires only that at least one remains. Positive volume in every node will ensure this.

The invariant must continue to hold when refinement removes an arc from an in-node DAG.

Lemma 5.1 (Effective intervals and sections)**.** *Replacing interval bounds by their effective endpoints leaves the feasible region unchanged. The region has positive volume if and only if every effective width is positive. When these widths are positive, fixing $X_x = c$ at any $c \in (\ell_x, r_x)$ leaves a positive-volume section in the remaining coordinates.*

Proof. If $y \ne x$ precedes x on a directed path, every feasible assignment satisfies $X_x > X_y > a_y$; the stored bound $X_x > a_x$ covers $y = x$. Taking the maximum over these bounds gives $X_x > \ell_x$. The upper-bound argument is the reverse. Thus all bounds given by (1) already hold.

Positive volume requires positive widths. Conversely, suppose all widths are positive. Along an arc $x \to y$, the definitions give $\ell_x \le \ell_y$ and $r_x \le r_y$. Choose numbers $0 < \theta_x < 1$ increasing in a topological order, and set

$$X_x = (1-\theta_x)\ell_x + \theta_x r_x.$$

Increasing θ_x to θ_y strictly increases this value because $r_x > \ell_x$. Replacing its endpoints by the larger endpoints ℓ_y, r_y cannot decrease it. Hence $X_x < X_y$. All coordinates are inside their

intervals and all arc inequalities hold strictly, so a sufficiently small open neighborhood of this assignment is feasible.

Now fix $X_x = c$ and delete its coordinate. The inequalities imposed by its incident arcs become upper bounds c at its in-neighbors and lower bounds c at its out-neighbors. Upper bounds propagate only backward along arcs, and lower bounds only forward. Any lower bound that previously reached a successor through x is superseded by $c > \ell_x$; any upper bound that reached a predecessor through x is superseded by $c < r_x$. Thus the new effective intervals are

$$\begin{cases} (\ell_y, \min(r_y, c)), & y \text{ precedes } x, \\ (\max(\ell_y, c), r_y), & y \text{ succeeds } x, \\ (\ell_y, r_y), & y \text{ is incomparable with } x. \end{cases}$$

The first two cases are disjoint because the in-node DAG is acyclic. A predecessor retains positive width: $r_y > \ell_y$ and $c > \ell_x \geq \ell_y$. A successor does too: $r_y > \ell_y$ and $c < r_x \leq r_y$. All other widths stay positive. The criterion just proved, applied to the remaining in-node DAG, shows that the section has positive volume. □

Lemma 5.2 (Refinement and child admission)**.** *Suppose the node regions have positive volume and the order invariant holds. Refining x at an interior anchor c preserves both properties, including when the child at c already exists. The transcript implies no relation between x and any label already in that child subtree.*

Proof. Consider first an existing child at c, and denote its label set by S. Choose a feasible assignment in the parent section $X_x = c$, with the other free coordinates pairwise distinct and different from every child anchor. Such an assignment exists by Lemma 5.1. Also choose feasible assignments in the other nodes.

There is a positive distance from c to every other child anchor and every free parent coordinate in this assignment. All incident inequalities at x are strict. Thus, with everything else fixed, x can be moved slightly to either side of c while keeping the old state feasible. The resulting two represented orders place x immediately before or immediately after the entire block S. Both orders respect the transcript. For each $y \in S$, one has $x < y$ and the other has $y < x$; neither relation can therefore be implied by the transcript.

Now take any order represented after the refinement. Deleting x from the child leaves an assignment of its old coordinates and hence an old child order. Apply the preceding perturbation using this assignment's parent coordinates and old child coordinates. It gives two feasible old orders that agree with the new one on every label except x. The new order inserts x somewhere into their child block. No answered comparison joins x to a label in S, so this insertion violates none of them. Relative to every label outside $S \cup \{x\}$, the whole block occupies the old anchor position. The item x is consequently on the same side of each outside label as in either of the two feasible old orders above. Comparisons between other labels are unchanged. The new order therefore respects the entire transcript.

If there was no child at c, the same argument uses an empty block: the new singleton child places x where its fixed parent coordinate stood. Finally, the remaining parent region is a positive-volume section, and the child receives an independent coordinate in $(0, 1)$. All node regions remain positive. □

5.2 The volume gained by refinement

Refining an item x stored at node v fixes $X_x = c$ in v and gives x a fresh unit coordinate in the child. To measure the volume change, let $S_x(c)$ be the volume of the section of K_v obtained by

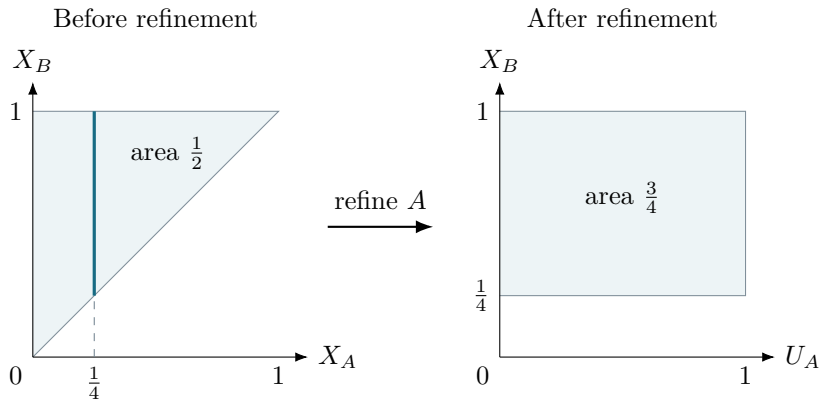


Figure 7: The highlighted section of the triangle has length 3/4. Replacing the fixed parent coordinate by a fresh unit coordinate in the child gives the rectangle on the right. Its area is 3/2 times the original area.

fixing $X_x = c$, measured in the remaining coordinates. Under the uniform distribution on K_v, the coordinate X_x has density

$$f_x(c) = \frac{S_x(c)}{V_v}, \qquad \mathbb{P}(a < X_x < b) = \int_a^b f_x(t)\,dt.$$

Lemma 5.3 (Refinement identity)**.** *Suppose every node region has positive volume. Refining x at an interior anchor c satisfies*

$$\frac{Z_{\text{after}}}{Z_{\text{before}}} = f_x(c). \tag{21}$$

Proof. Let u be the child at c, taking $V_u = 1$ if it is newly created. The parent loses the coordinate X_x and retains precisely the section of volume $S_x(c)$. The child gains a new coordinate $U_x \in (0,1)$, with no constraints involving its old coordinates. Its region becomes $K_u \times (0,1)$ and still has volume V_u. Therefore the affected factors in the product change as follows:

$$V_v V_u \quad \longrightarrow \quad S_x(c) V_u.$$

Their ratio is $S_x(c)/V_v = f_x(c)$. All other node factors stay unchanged. The dimension lost in the parent is restored by the new child coordinate. □

Thus the refinement's volume gain is $g = \log f_x(c)$.

For example, the parent region $0 < X_A < X_B < 1$ has area 1/2. Fixing $X_A = 1/4$ leaves the interval $X_B \in (1/4, 1)$ of length 3/4. The new child coordinate $U_A \in (0,1)$ turns this into a rectangle of area 3/4, so refinement multiplies Z by 3/2; see Figure 7. Both regions represent the order $A < B$. Replacing the fixed parent coordinate by a fresh child coordinate gives the larger area.

5.3 Choosing an anchor

The choice of anchor balances two quantities: a larger density gives more volume gain, while a smaller anchor depth costs less in (14). We will bound the density using the number of coordinates comparable with x. Put

$$B = (k+1)^{2H+1}. \tag{22}$$

In an in-node DAG of degree at most k and directed paths of at most $2H$ vertices, every coordinate comparable with x lies within undirected distance $2H$ of x. Thus B bounds the number of other coordinates comparable with x.

Set the calibration width

$$w_* = 2^{-2kB}. \tag{23}$$

The finite calculations use this scale to guarantee success when a queried coordinate has a sufficiently large effective interval.

Lemma 5.4 (When certification succeeds)**.** *Suppose a node has positive feasible volume, its in-node DAG has degree at most k, and its directed paths have at most $2H$ vertices. If any free coordinate involved in a comparison has effective width at least w_*, comparison certification succeeds. If $W_x \geq w_*$, then* `CentralInterval`(v, x) *succeeds.*

Section 6 proves this lemma. In particular, failure of a central-interval certificate implies $W_x < w_*$. This is the information used to justify the anchor routine's fallback.

Lemma 5.5 (Anchor selection)**.** *Suppose a node's feasible region has positive volume, its interval endpoints are dyadic, and its in-node DAG has degree at most k and directed paths of at most $2H$ vertices. With calibration width (23),* `ChooseAnchor` *deterministically returns an anchor $c \in (\ell_x, r_x)$ at depth m in the implicit anchor tree satisfying*

$$\log f_x(c) \geq (1 - 1/k)m - 4. \tag{24}$$

If x is refined because comparison certification failed, then

$$\log f_x(c) > k. \tag{25}$$

Algorithm 3 first tries `CentralInterval`. On success, it uses the returned interval; on failure, it uses the middle half of the effective interval. It then selects the shallowest anchor in the chosen interval. Both choices give the density bound required by Lemma 5.5.

Algorithm 3: `ChooseAnchor`(x): choose a position for refinement.

Input: A free item x stored at node v and its effective interval (ℓ_x, r_x).
Output: A dyadic anchor $c \in (\ell_x, r_x)$.

```
J ← CentralInterval(v, x)
if J is FAIL then
    W_x ← r_x − ℓ_x
    J ← [ℓ_x + W_x/4, r_x − W_x/4]
[a, b] ← J
u ← the root of the implicit anchor tree
while true do
    if c(u) < a then
        u ← the right child of u
    else if c(u) > b then
        u ← the left child of u
    else
        return c(u)
```

An anchor at depth m has $m+1$ fractional binary digits in reduced form. Thus the walk finds the shortest binary fraction in $[a, b]$.

Lemma 5.6 (Shortest anchor in an interval)**.** *For a nondegenerate closed interval $[a, b] \subset (0, 1)$, the walk in Algorithm 3 returns its unique minimum-depth anchor. If its node is u and its depth is m, then $[a, b] \subseteq I(u)$ and $|I(u)| = 2^{-m}$.*

Proof. Initially $[a, b] \subseteq I(u) = [0, 1]$. Whenever $c(u) < a$, the whole interval lies in the right child's interval; whenever $c(u) > b$, it lies in the left child's interval. The rejected anchor and the

other subtree contain no candidate in $[a, b]$. The first accepted anchor therefore has minimum depth, and every other anchor of that depth lies outside $I(u)$. The intervals halve at every step and continue to contain $[a, b]$, so the walk must accept before their length becomes smaller than $b - a$. □

5.3.1 Density bounds for central intervals

The first bounds use the effective width and the number of comparable coordinates.

Lemma 5.7 (Coordinate density bounds)**.** *Suppose a node's feasible region has positive volume, and x has at most B other comparable coordinates and effective interval (ℓ, r) of width W. Its density satisfies*

$$f_x(t) \le \frac{B+1}{W}, \tag{26}$$

$$f_x(t) \ge \frac{4^{-B}}{W} \qquad (\ell + W/4 \le t \le r - W/4). \tag{27}$$

The lower bound controls the fallback choice in the middle half of the effective interval. The upper bound will help bound the probability of an unresolved comparison. Appendix A proves both by comparing convex sections.

For a certified central interval we use another consequence of convexity. A density is *log-concave* if its positive support is convex and its logarithm is concave there. The uniform density on a convex feasible region is log-concave, and Prékopa's marginal theorem gives the same property to each coordinate density [11, Theorem 8]. The following bound says that a position with substantial probability on each side has density comparable to the largest value.

Lemma 5.8 (Density at central positions)**.** *Let X have a log-concave density f. If $\mathbb{P}(X \le c) \ge p$ and $\mathbb{P}(X \ge c) \ge p$ for some $0 < p \le 1/2$, then*

$$f(c) \ge p \sup_t f(t).$$

Appendix A proves the lemma directly from log-concavity.

5.3.2 Anchors from a certified interval

Lemma 5.9 (Certified anchor choices)**.** *If* `CentralInterval` *succeeds, the anchor c selected by Algorithm 3 has depth m and satisfies*

$$\log f_x(c) \ge m - 4, \qquad \log f_x(c) \ge \log(1/W_x) - 2.$$

Proof. Let J be the interval returned by `CentralInterval`, and put $M = \sup_t f_x(t)$. By Lemma 3.2, J is contained in (ℓ_x, r_x) and has probability at least $1/4$, so $M|J| \ge 1/4$. Every position $c \in J$ also has probability at least $1/4$ on each side. Lemma 5.8 therefore gives $f_x(c) \ge M/4$ throughout J.

Lemma 5.6 selects $c \in J$ at a node u whose interval contains J. Its interval has length 2^{-m}, so

$$2^{-m} f_x(c) \ge |J| \frac{M}{4} \ge \frac{1}{16}.$$

This proves the first bound. Since f_x integrates to one over an interval of width W_x, its supremum satisfies $M \ge 1/W_x$. The inequality $f_x(c) \ge M/4$ proves the second bound. □

5.3.3 The anchor guarantee

The certified choice already gives the required bound. For the fallback, failure of the central-interval calculation implies $W_x < w_*$, so $\log(1/W_x) > 2kB$. This pays for the loss of $2B$ in the lower density bound (27).

Proof of Lemma 5.5. The in-node DAG bounds imply that x has at most B other comparable coordinates, by (22). If central certification succeeds, Lemma 5.9 gives the stronger lower bound $m-4$. Otherwise, Lemma 5.4 gives $W_x < w_*$. Write $L = \log(1/W_x) > 2kB$. The fallback anchor is in the central half of its effective interval, so (27) gives

$$\log f_x(c) \geq L - 2B.$$

By Lemma 5.6, its node interval contains that central half. Hence $W_x/2 \leq 2^{-m}$ and $m \leq L+1$, giving

$$\log f_x(c) \geq L - 2B > (1-1/k)L \geq (1-1/k)(m-1) > (1-1/k)m - 4.$$

This proves (24) in both branches.

Now suppose comparison certification failed and the algorithm selected x for refinement. Lemma 5.4 implies $W_x < w_*$, regardless of the subsequent central-interval result. If central certification succeeds, Lemma 5.9 gives

$$\log f_x(c) \geq \log(1/W_x) - 2 > 2kB - 2 > k.$$

If it fails, the fallback bound above gives $\log f_x(c) \geq \log(1/W_x) - 2B > 2B(k-1) > k$. This proves (25). Both cases use a finite deterministic calculation to obtain the central interval, followed by the finite walk of Lemma 5.6. □

5.4 Bounds during a comparison call

The preparation loop in Algorithm 1 simplifies the current node before an answer is selected. We check that the in-node DAG bounds hold throughout the call, and that the loop makes only finitely many moves.

Lemma 5.10 (In-node DAG bounds and termination)**.** *Suppose every node region has positive volume and every in-node DAG has degree at most k and directed paths of fewer than $2H$ vertices at the start of a call. These properties hold throughout the call. Each label is refined at most once, so the call makes at most n refinements.*

Proof. A refinement uses an interior anchor by Lemma 5.5. It leaves a positive-volume parent section by Lemma 5.1 and gives the child a fresh unit coordinate. It deletes a vertex and its incident arcs from the parent and adds an isolated vertex to the child, so it cannot increase any degree or extend any directed path.

A returned comparison certificate gives its chosen answer positive probability, so recording it preserves positive feasible volume. Its arc cannot create a directed cycle, which would make the region empty. Immediately before a new answer is selected, the current in-node DAG has no path on H vertices and the queried free vertices have degree below k. Adding their comparison arc leaves degree at most k. Any new directed path consists of two old paths joined by this arc, so it has fewer than $2H$ vertices. An answer involving an anchor adds only an interval bound. These observations prove positivity and the in-node DAG bounds by induction through the call.

Once a label is refined, it has an isolated coordinate with interval $(0,1)$ in its child. Until the final answer, every subsequent state change is another refinement. These moves affect interval bounds only at neighbors of the vertex being removed, so they cannot narrow this isolated coordinate or give it an incident arc. It cannot lie on a selected path or have degree k. Lemma 3.1

guarantees that comparison certification succeeds whenever such a unit-interval coordinate is queried, so it cannot be selected after a failed certificate either. Thus no label is selected again, even if the query is rerouted to its new node. Every restart refines at least one label, giving the bound of n refinements. □

Proof of Proposition 4.2. Initially every node region has positive volume, every order is represented, and the in-node DAG bounds hold. We induct over the changes made by a comparison call. The bounds follow from Lemma 5.10. At a refinement, Lemma 5.5 supplies an interior anchor, and Lemma 5.2 preserves positive volume and the order invariant. This applies to every item of a selected path, including those whose incident arcs have already been removed by earlier moves.

If the queried labels lie in different child subtrees, their anchors determine the answer in every represented order. Otherwise, a comparison certificate supplies a chosen answer retaining at least $1/2 - \eta > 0$ of the current node's volume, by the acceptance test in Algorithm 1 and Lemma 3.1. Recording its arc or interval bound makes that answer true in every represented order and preserves every earlier constraint. This completes the induction.

The comparison calculation and anchor search terminate by Lemmas 3.1 and 5.5. Together with Lemma 5.10, this proves termination of the full call. These analytic lemmas apply to arbitrary positive-volume regions with the stated in-node DAG bounds; their proofs are independent of the adversary's history. □

5.5 Distinct children for a path batch

Proof of Lemma 4.5. Every in-node arc on the selected path records an answered comparison. The transcript therefore implies an order between every pair of labels on the path. Those implications remain true after the in-node arcs are deleted. By Lemma 5.2, two of these labels cannot be admitted to the same child of their parent. The procedure completes the entire selected batch in this parent, so all its destinations are distinct. □

5.6 The refinement guarantee

We finish by relating an anchor's child-choice cost to the volume change it produces. The density guarantee controls its depth, while the chosen weights make the cost almost proportional to that depth.

Proof of Lemma 4.4. The refinement identity gives $g = \log f_x(c)$. If the selected anchor has depth m, the anchor guarantee (24) implies

$$g \geq -4, \qquad m \leq \frac{g+4}{1-1/k}.$$

Consequently, (14) gives

$$\ell(c) \leq \frac{\gamma}{1-1/k}(g+4) + \log k + 1.$$

For $k \geq 64$, the estimate $-\log(1-1/k) \leq 1/((k-1)\ln 2) \leq 3/(2k)$ gives $\gamma/(1-1/k) \leq 1 + 3/k$. We may therefore choose

$$a = 1 + \frac{3}{k}, \qquad b = 4a + \log k + 1.$$

Because $g + 4 \geq 0$, increasing its coefficient to a preserves the inequality, including when g is negative. This proves $\ell(c) \leq ag + b$. The chosen coefficients satisfy $a \geq 1$ and $1 \leq b \leq \log k + 6 < k + 1$, as well as the stated asymptotic bounds. If the refinement follows unsuccessful comparison certification, (25) gives $g > k$. □

6 Computing comparison certificates

We implement `Certify` and `CentralInterval` by computing finite lists of enclosing intervals for the queried positions. Disjoint intervals certify a comparison. Counts of intervals lying to the left, inside, and to the right of a proposed central interval certify its three probability bounds.

6.1 A coordinate update with finitely many choices

Let G be the in-node DAG. Compute its effective endpoints ℓ_z, r_z using the full graph, in temporary storage. For a current assignment Y, the interval permitted at coordinate z has endpoints

$$L_z = \max\big(\{\ell_z\} \cup \{Y_u : u \to z\}\big), \qquad U_z = \min\big(\{r_z\} \cup \{Y_u : z \to u\}\big).$$

Fix a power of two S. For a digit $j \in \{0, \dots, S-1\}$, update

$$Y_z \leftarrow \frac{Y_z + (S-1-j)L_z + jU_z}{S}. \tag{28}$$

The coefficients are nonnegative and sum to one. Thus an assignment satisfying the interval bounds and arc inequalities weakly continues to do so after an update, including when $L_z = U_z$.

Choosing the digit uniformly preserves the uniform distribution on the feasible region. Start from a uniform feasible assignment and condition on all coordinates other than z. Then Y_z is uniform on (L_z, U_z). Its normalized position $r = (Y_z - L_z)/(U_z - L_z)$ is sent to

$$r' = \frac{r + j}{S}.$$

For a fixed digit, this is uniform on the corresponding one of the S equal subintervals. Averaging over the digits makes r' uniform on $(0, 1)$ again.

6.2 Enclosing intervals from two calculations

Fix a topological order of the full graph G, and let T be the set of queried coordinates. Retain every vertex within undirected distance ρ of T, together with all arcs between retained vertices. Copy their effective endpoints from the full graph. Vertices at distance exactly ρ form the boundary; those at smaller distance form the interior.

Run two calculations using the same digits. The lower calculation starts at $A_z = \ell_z$, and the upper calculation starts at $B_z = r_z$. Keep the boundary values fixed throughout both calculations. A *sweep* updates each interior coordinate once, in the restriction of the full topological order. All neighbors of an updated vertex are retained.

Enumerate every assignment of digits to t sweeps. If there are b' interior vertices, this gives

$$N = S^{b't} \tag{29}$$

computations. Each contributes the intervals $[A_x, B_x]$, $x \in T$, to the list, including repeated entries.

Algorithm 4: `Enclose`: compute a finite list of enclosing intervals.

Input: The retained in-node DAG and its effective bounds, its interior vertices, the restricted full topological order, queried coordinates T, and integers S, t.
Output: A list of enclosing intervals $([A_x, B_x])_{x\in T}$, one per digit assignment.
$\mathcal{L} \leftarrow$ the empty list
foreach *assignment of digits* $j_{z,s} \in \{0, \dots, S-1\}$ *to the* $b't$ *interior visits* **do**
  $A_z \leftarrow \ell_z$ and $B_z \leftarrow r_z$ for every retained vertex z
  for $s = 1, \dots, t$ **do**
    foreach *interior vertex* z *in the supplied order* **do**
      Apply (28) to A_z, using the current A values and digit $j_{z,s}$
      Apply (28) to B_z, using the current B values and the same digit
  Append $([A_x, B_x])_{x\in T}$ to $\mathcal{L}$
return $\mathcal{L}$

The intervals have an exact probabilistic interpretation. A *coupling* is a joint distribution with specified marginal distributions; here it joins a uniformly selected list entry to a uniform feasible assignment.

Theorem 6.1 (Local enclosing intervals)**.** *Let $d, h, r \geq 1$ be integers and let $\delta, \alpha \in (0,1)$ be rational. There are computable parameters S, t, ρ and a bound E, depending only on d, h, r, δ, α, with the following property.*

Suppose G is a finite DAG with dyadic interval bounds, degree at most d, directed paths of at most h vertices, and a positive-volume feasible region $K \subseteq (0,1)^{V(G)}$. For any nonempty set T of at most r coordinates, Algorithm 4 returns a list satisfying two guarantees:

1. *A uniformly selected entry can be coupled with X uniform on K so that $A_x \leq X_x \leq B_x$ for every $x \in T$.*
2. *At most a fraction α of its entries have $\max_{x\in T}(B_x - A_x) > \delta$.*

The calculation enumerates at most E digit assignments. If the stored bounds have at most Λ fractional binary digits, every computed endpoint has at most $\Lambda + \log E$ fractional digits.

The enclosure guarantee holds for every finite number of sweeps and every choice of radius $\rho \geq 1$. Start a full assignment uniformly on K and update the entire graph with independent uniform digits. Share its digits with the two endpoint calculations at interior vertices. The update is increasing in every coordinate, since maxima, minima, and the nonnegative linear combination in (28) are increasing. Thus it preserves the inequalities $A_z \leq X_z \leq B_z$ at an interior update. At a boundary vertex the full position always remains between its fixed effective endpoints, even as it changes during the full calculation. The full assignment remains uniform, and every interior digit sequence has probability $1/N$. This proves the first guarantee.

To make most intervals narrow, we use the degree and path-length bounds. Following dependencies on newly updated predecessors reaches a source within h vertices, where the lower endpoint is fixed. Most digit choices therefore reduce the interval widths. Bounded degree limits the number of dependencies to consider across successive sweeps. Appendix B proves the width bound and gives explicit parameter choices.

6.3 Certifying comparison answers

For two queried coordinates, let p be the fraction of entries with $B_x < A_y$, and let q be the fraction with $B_y < A_x$. The routine `Certify` always returns these two counts divided by N. For a threshold comparison, use the fixed interval $[c, c]$ for the threshold. The comparison rule accepts an answer when its certified fraction is at least $1/2 - \eta$; otherwise it refines a queried free item.

Proof of Lemma 3.1. The two certification conditions are disjoint, so $p, q \geq 0$ and $p + q \leq 1$. Whenever $B_x < A_y$, enclosure gives $X_x < X_y$; the reverse condition gives the reverse order. Consequently

$$p \leq \mathbb{P}(X_x < X_y), \qquad q \leq \mathbb{P}(X_y < X_x).$$

The same argument applies to a fixed threshold. These bounds hold for every list, including when neither fraction passes the acceptance test. The counts and their denominators are exact integers, so the routine is deterministic. The unit-width success guarantee follows from Lemma 5.4, proved below, since $w_* \leq 1$. □

6.4 Certifying a central interval

For a free coordinate x, `CentralInterval` runs Algorithm 4 with $T = \{x\}$. Write its intervals as $[A_i, B_i]$, $1 \leq i \leq N$. The chosen S is divisible by four, so $N/4$ is an integer. Sort the upper endpoints and take their $N/4$-th value as u. Sort the lower endpoints and take their $3N/4$-th value as v. Count the intervals wholly contained in $[u, v]$. If $u \geq v$ or fewer than $N/4$ intervals pass this test, return `FAIL`. Otherwise return $[u, v]$.

Proof of Lemma 3.2. At least $N/4$ intervals end at or below u, and at least $N/4$ begin at or above v. On success, at least $N/4$ intervals also lie wholly inside $[u, v]$. Enclosure certifies mass at least $1/4$ in each of the three regions. The true distribution is continuous and supported on (ℓ_x, r_x), so these masses imply $\ell_x < u < v < r_x$. The endpoints are dyadic, and sorting and counting finite lists are deterministic. □

6.5 When certification succeeds

The preceding certificates are valid whenever their finite tests pass. To prove that the tests succeed on sufficiently dispersed coordinates, we bound the fraction of entries whose intervals overlap. The following geometric estimate applies to every pair of distinct free coordinates.

Lemma 6.2 (A bound for small differences)**.** *Let X be uniform on a positive-volume node region, and suppose x has at most B other comparable coordinates and effective width W. Let Y be a fixed number or X_y for any distinct free coordinate y. For every interval I of length a,*

$$\mathbb{P}(X_x - Y \in I) \leq \frac{(B+1)a}{W}. \tag{30}$$

Appendix A proves this bound by applying a density estimate for linear projections to a convex region with at most $B + 1$ free coordinates.

Proof of Lemma 5.4. Computing effective endpoints preserves the positive-volume feasible region by Lemma 5.1. Calibrate both routines using Theorem 6.1 with

$$d = k, \qquad h = 2H, \qquad r = 2, \qquad \alpha = \frac{\eta}{2}, \qquad \delta = \frac{\eta w_*}{8(B+1)}. \tag{31}$$

These parameters are fixed for the interaction. They do not depend on the queried widths.

Suppose one queried free coordinate x has $W_x \geq w_*$. If the computed intervals overlap and both have width at most δ, the enclosed positions differ by at most 2δ. Lemma 6.2 therefore bounds the unresolved fraction by

$$1 - p - q \leq \alpha + \mathbb{P}(|X_x - X_y| \leq 2\delta) \leq \alpha + \frac{4(B+1)\delta}{w_*} = \eta.$$

Thus $\max(p, q) \geq (1 - \eta)/2 > 1/2 - \eta$, so comparison certification succeeds. For a threshold c, an unresolved interval contains c. If its width is at most δ, its enclosed position is within δ of c, giving the stronger bound

$$\alpha + \frac{2(B+1)\delta}{w_*} = \frac{3\eta}{4}. \tag{32}$$

It remains to show that `CentralInterval` succeeds when $W_x \geq w_*$. By (32), at most a fraction $\tau = 3\eta/4$ of its intervals contain any fixed real number. This bound applies in particular to the selected endpoints u and v. An interval extending below u either ends strictly below u or contains u. By the rank defining u, fewer than $N/4$ intervals end strictly below it. Thus at most $(1/4 + \tau)N$ intervals extend below u. Similarly, at most $N/4$ intervals begin strictly above v, so at most $(1/4 + \tau)N$ extend above v.

At least $(1/2 - 2\tau)N \geq N/4$ intervals therefore lie wholly inside $[u, v]$, since $\eta < 1/32$. Enclosure gives this interval mass at least $1/4$. Continuity forces $u < v$, so both success tests pass. □

The next section chooses the parameters as a function of n and bounds the work in every comparison call.

7 A uniform polynomial-time implementation

For each fixed degree threshold k, the preceding sections give a finite deterministic adversary. We now bound how long it maintains the recursive state and choose k slowly enough that its local calculations take polynomial time. Computation is measured on a fixed deterministic multitape Turing machine, with integers represented in binary.

7.1 A finite horizon for the recursive state

Recall the horizon $Q(n) = n \lceil \log n \rceil$ from (2). The algorithm runs the recursive adversary for at most $Q(n)$ answers, recording this prefix of the transcript. If another query arrives, it forms the directed graph of those answers and chooses its lexicographically first topological ordering. It answers that query and every subsequent query from the resulting fixed ranks.

Proposition 4.2 ensures that the prefix is consistent with a total order, so this graph is acyclic and the topological ordering exists. Every later answer remains consistent with the prefix. If the sorter reaches the horizon, it has already made at least $n \log n$ comparisons. If it stops earlier, the comparison bound for the recursive adversary applies.

Lemma 5.10 bounds the number of refinements in one call by n. Thus their total number P, including those in a call still in progress before the horizon, satisfies

$$P \leq nQ(n) = O(n^2 \log n). \tag{33}$$

The recursive tree has at most $P + 1$ nodes and contains n item records. The stored transcript has at most $Q(n)$ pairs. After switching to fixed ranks, the recursive state is no longer needed.

7.2 An explicit budget for the finite calculations

Use the calibration in (31) for the local enclosure theorem. Write S_k for its number of digit choices, t_k for the number of sweeps, and b_k for the bound on retained vertices in (45). The number of digit sequences in one calculation is at most

$$E_k = S_k^{b_k t_k}. \tag{34}$$

For $k = 64, 128, \ldots$, a fixed program writes the complete parameter tuple and E_k in binary. On input size n, simulate this program for n steps. Among the tuples it has completed, choose the largest k with

$$E_k \le n. \tag{35}$$

Use these parameters for the entire recursive phase. If no tuple qualifies, answer all comparisons according to the fixed label order.

The initialization takes polynomial time: the simulation writes at most n bits, and its completed tuples can be parsed and compared within a polynomial bound. For every fixed k, the program finishes its tuple in some finite time $T(k)$. Once $n \ge \max\{T(k), E_k\}$, that tuple is available and meets the budget. Hence the selected $k(n)$ tends to infinity.

7.3 Precision and work per call

Let Λ be the largest fractional binary precision among the stored bounds and anchors. The finite calculation produces endpoints with at most $\Lambda + \log E_k$ digits. If central certification succeeds, either endpoint of the returned interval is an admissible anchor candidate. The shortest binary fraction in that interval uses no more digits. If certification fails, the midpoint of the effective interval is a candidate in its middle half and uses at most $\Lambda + 1$ digits. Since $E_k \ge 2$, either branch increases the stored precision by at most $\log E_k$. Maxima and minima of existing bounds add no precision. Starting from the bounds $0, 1$, we obtain

$$\Lambda \le P \log E_k = O(n^2 \log^2 n). \tag{36}$$

There are at most n digit sequences per certificate calculation. Each sequence uses two endpoint calculations, each with at most $b_k t_k \le \log n$ updates. Each calculation adds at most $b_k t_k \log S_k = \log E_k \le \log n$ fractional bits. The frozen boundary requires no updates. Tallies have at most $\log n + 1$ bits. Central certification sorts at most n lower and upper endpoints and counts the intervals inside the proposed interval. All this arithmetic involves polynomially many bits. The shortest-fraction search examines at most the bit length of the candidate just described, so it also takes polynomial time.

A comparison call makes at most n refinements and hence at most $n + 1$ calls to `Certify` and n calls to `CentralInterval`. Routing queries and computing effective endpoints take polynomial time in the stored state. Directed-path lengths are computed in reverse topological order; if $H > n$, no path can require refinement. The lexicographic tie rules can be implemented by scanning labels in order. Recording the bounded transcript and computing its topological ordering also take polynomial time. After the horizon, each answer compares two stored ranks. Thus one fixed polynomial bounds the work per query, independently of the selected parameters and the length of the interaction.

Proof of Theorem 1.1. Initialization and every comparison call are deterministic and take polynomial time. The recursive phase is legal by Proposition 4.2, and the final fixed order respects its recorded answers. For all sufficiently large n, a parameter tuple qualifies. If the sorter reaches $Q(n)$ answers, it has already made at least $n \log n$ comparisons. Otherwise Theorem 4.1 gives

$$q \ge \frac{\log(n!) - n}{1 + O(\log k(n)/k(n))} = (1 - o(1)) n \log n,$$

since $k(n) \to \infty$. The fixed-order rule used for the remaining input sizes is deterministic and legal as well. □

A Coordinate densities

We prove the density bounds used to choose anchors and to bound the fraction of unresolved comparisons. The geometric argument applies to linear projections of convex regions. Let V be the set of coordinates, let $N = |V|$, and let $K \subseteq \mathbb{R}^V$ be their feasible region. For a coordinate x and a real number t, write

$$K_x(t) = \{(z_i)_{i \neq x} : z \in K,\ z_x = t\}, \qquad S_x(t) = \operatorname{vol}_{N-1} K_x(t).$$

Here $K_x(t)$ is the $(N-1)$-dimensional section obtained by fixing coordinate x at t. The uniform distribution on K therefore gives x the density

$$f_x(t) = \frac{S_x(t)}{\operatorname{vol}_N K}.$$

These formulas define f_x pointwise, and we use this definition throughout. If some coordinates have already been fixed, we use the same definitions in the remaining free coordinates. In dimension zero, the volume of the singleton consisting of the empty assignment is one.

Lemma A.1 (Density of a linear projection)**.** *Let $C \subseteq \mathbb{R}^d$ be a bounded open convex region of positive volume, and let φ be a nonzero linear functional with $\varphi(C) = (\ell, r)$ of width W. For X uniform on C, the density g of $\varphi(X)$ satisfies*

$$g(t) \le \frac{d}{W}, \qquad g(t) \ge \frac{4^{-(d-1)}}{W} \quad (\ell + W/4 \le t \le r - W/4).$$

Proof. An invertible linear change of coordinates makes φ the first coordinate. Its constant Jacobian preserves uniformity on the transformed region, so it suffices to prove the bounds for a coordinate projection. Write $S(t)$ for the section volume at that coordinate's value t; then $g(t) = S(t)/\operatorname{vol} C$.

Choose points in $\overline{C}$ whose first coordinates are ℓ and r. Such points exist because C is bounded. For $\ell < t < s < r$, join every point of the section at t to the point at r. Since C is open and convex, the image with first coordinate s lies in the section of C there. Each of its $d-1$ directions is scaled by $(r-s)/(r-t)$, so

$$S(s) \ge \left(\frac{r-s}{r-t}\right)^{d-1} S(t) \qquad (t < s). \tag{37}$$

Joining instead to the point at ℓ gives

$$S(s) \ge \left(\frac{s-\ell}{t-\ell}\right)^{d-1} S(t) \qquad (s < t). \tag{38}$$

Normalize these inequalities and integrate over s on both sides of t:

$$1 \ge g(t) \left(\int_\ell^t \left(\frac{s-\ell}{t-\ell}\right)^{d-1} ds + \int_t^r \left(\frac{r-s}{r-t}\right)^{d-1} ds \right) = \frac{W g(t)}{d}.$$

Thus $g(t) \le d/W$.

For the lower bound, choose t_0 with $g(t_0) \ge 1/W$, which exists because g integrates to one over an interval of length W. For any $s \in [\ell + W/4, r - W/4]$, the appropriate ratio in (37) or (38) is at least $1/4$. Consequently

$$g(s) \ge 4^{-(d-1)} g(t_0) \ge \frac{4^{-(d-1)}}{W}.$$

The case $s = t_0$ satisfies the same bound directly. □

Proof of Lemma 5.7. Fix all coordinates incomparable with x, at values leaving a nonempty region. By the section formula in Lemma 5.1, each such fixing leaves x's effective interval unchanged. The remaining region C is open and convex, has dimension $d \le B+1$, and still projects onto (ℓ, r) in coordinate x. Lemma A.1 therefore gives both required bounds for the density of x in C.

Average over the fixed coordinates. If ω denotes their values, C_ω the remaining region, and g_ω its coordinate density, integration of each section gives the pointwise identity

$$f_x(t) = \frac{\int \mathrm{vol}(C_\omega) g_\omega(t)\, d\omega}{\int \mathrm{vol}(C_\omega)\, d\omega}.$$

Empty slices contribute zero. The bounds on g_ω are uniform in ω, so they also hold for f_x. □

Proof of Lemma 6.2. For a fixed number Y, integrate the coordinate-density upper bound over $I+Y$. Now let $Y = X_y$ for any free coordinate $y \neq x$. As above, fix all coordinates incomparable with x. The remaining convex region C has dimension $d \le B+1$, and x retains its effective width W. We show that the projection $X_x - X_y$ has width at least W in each such region.

If y is incomparable with x, it is now fixed, so the projection is a translate of X_x and has width W. Otherwise y remains free. Write (ℓ_z, r_z) for effective intervals in C; x's endpoints are unchanged. If a directed path goes from x to y, then $r_y \ge r_x$. Fixing $X_y = t \in (\ell_y, r_y)$ leaves x the interval $(\ell_x, \min(r_x, t))$, by Lemma 5.1. As t approaches r_y, its width tends to W. On that section the difference $X_x - t$ varies over an interval of the same width. Thus the projection of the whole region C has width at least W. If a directed path goes from y to x, let t approach ℓ_y instead. Here $\ell_y \le \ell_x$, and the remaining interval $(\max(\ell_x, t), r_x)$ again has width tending to W.

Lemma A.1 bounds the conditional density of the difference by $d/W \le (B+1)/W$. Integrating over I and averaging over the fixed coordinates proves (30). If both coordinates have at most B comparable coordinates, exchanging x and y gives the same bound with $\max(W_x, W_y)$ in the denominator. □

The coordinate densities above are log-concave pointwise: the uniform density on K is log-concave, and integrating out the other coordinates preserves this property by Prékopa's marginal theorem [11, Theorem 8]. We finish with the density bound used for positions having substantial probability on each side.

Proof of Lemma 5.8. For $s \le c \le t$, log-concavity gives

$$f(s)f(t) \le f(c)f(s+t-c).$$

Indeed, c and $s+t-c$ are complementary convex combinations of s, t; multiply the two log-concavity inequalities. Fixing $t \ge c$ and integrating over $s \le c$ gives

$$pf(t) \le f(c) \int_{-\infty}^{c} f(s+t-c)\, ds \le f(c).$$

Here the first inequality uses the mass at least p to the left of c, and the last uses that a density integrates to one. Fixing $s \le c$ and integrating over $t \ge c$ instead uses the right-hand mass and gives $pf(s) \le f(c)$. Taking the supremum over both sides of c proves the claim. □

B Why the enclosing intervals become narrow

The enclosure part of Theorem 6.1 was proved in Section 6. Here we bound the fraction of digit sequences that leave a wide interval at a queried coordinate. We compare the lower and upper calculations with each other. Their initial widths are at most one, and their boundary values stay fixed. For the analysis, choose each interior digit independently and uniformly; each enumerated sequence then has probability $1/N$.

B.1 The decrease after one sweep

Write $\Delta_z = B_z - A_z$. Superscripts old and new refer to the start of a sweep and the value after a coordinate's update. For an interior vertex z, the shared digit j_z gives

$$\Delta_z^{\text{new}} \le \frac{1}{S}\Delta_z^{\text{old}} + \frac{S-1-j_z}{S}\max_{u\to z}\Delta_u^{\text{new}} + \frac{j_z}{S}\max_{z\to u}\Delta_u^{\text{old}}, \tag{39}$$

with an empty maximum equal to zero. The effective interval bounds in the two calculations are identical.

Let $B_h(z)$ be the vertices at undirected distance at most h from z, and suppose they are all interior vertices. Put $D = \max_{u\in B_h(z)}\Delta_u^{\text{old}}$ and $a_u = (S-1-j_u)/S$. At z, (39) gives

$$\Delta_z^{\text{new}} \le (1-a_z)D + a_z\max_{u\to z}\Delta_u^{\text{new}}.$$

Follow a predecessor attaining the maximum, and continue backwards to a source. This path has at most h vertices; all its vertices and their successors lie in $B_h(z)$. At the source the predecessor term is zero. Substitution along the path bounds the new width by $\left(1-\prod_u a_u\right)D$.

Write $\Delta_z(s)$ for the width after s sweeps. Call the pair (z,s) *bad* if, during sweep s, the digit at z or any vertex preceding z on a directed path equals $S-1$. Otherwise all factors a_u in the preceding expansion are at least $1/S$, and there are at most h of them. Put $\beta = 1-S^{-h}$. Consequently,

$$\Delta_z(s) \le \kappa_z(s)\max_{u\in B_h(z)}\Delta_u(s-1), \qquad \kappa_z(s) = \begin{cases} 1, & (z,s) \text{ is bad,} \\ \beta, & \text{otherwise.} \end{cases} \tag{40}$$

B.2 Averaging powers of the widths

We must control a maximum over several earlier widths. Taking a power and bounding the maximum by a sum makes this possible when the expected contraction is strong enough. The following lemma also allows the set of vertices under consideration to shrink after each sweep.

Lemma B.1 (Averaging contraction)**.** *Let $V_0 \supseteq \cdots \supseteq V_t$, and let nonnegative quantities $D_z(s)$ be determined by the first s sweeps. Suppose $D_z(0) \le 1$ and, for $z \in V_s$, there are nonnegative multipliers $\kappa_z(s)$ with*

$$D_z(s) \le \kappa_z(s)\max_{u\in N(z)} D_u(s-1), \qquad N(z)\subseteq V_{s-1}, \quad 1 \le |N(z)| \le M.$$

If, for an integer $p \ge 1$, the conditional expectation of $\kappa_z(s)^p$ given all earlier sweeps is at most $1/(2M)$, then for $z \in V_s$ and every $\delta > 0$,

$$\mathbb{E}D_z(s)^p \le 2^{-s}, \qquad \mathbb{P}(D_z(s) > \delta) \le 2^{-s}\delta^{-p}. \tag{41}$$

Proof. The old widths are fixed when conditioning on the earlier sweeps. Since the maximum of their p-th powers is at most their sum,

$$\mathbb{E}D_z(s)^p \le \frac{1}{2M}\sum_{u\in N(z)}\mathbb{E}D_u(s-1)^p \le \frac{1}{2}\max_{u\in N(z)}\mathbb{E}D_u(s-1)^p.$$

Induction gives the moment bound. The probability bound follows by Markov's inequality. □

B.3 Choosing the parameters

Both $B_h(z)$ and the set consisting of z and its directed-path predecessors have at most M vertices. Choose

$$M = (d+1)^{h+1}, \qquad S = \text{the least power of two at least } 4M^2. \tag{42}$$

The bad event in (40) involves only fresh digits from the current sweep. Its conditional probability is at most $M/S \le 1/(4M)$. With $\beta = 1 - S^{-h}$, choose the smallest positive integer p satisfying the rational test

$$\beta^p \le \frac{1}{4M}. \tag{43}$$

Consequently,

$$\mathbb{E}[\kappa_z(s)^p \mid \text{earlier sweeps}] \le \frac{1}{4M} + \beta^p \le \frac{1}{2M}.$$

This estimate uses all directed-path predecessors in defining the bad event, so it also covers the predecessor choices made in the one-sweep expansion. No independence between vertices within a sweep is required.

Choose the smallest positive integer t satisfying

$$2^{-t} \le \frac{\alpha\delta^p}{r}, \qquad \rho = ht + 1. \tag{44}$$

These choices are computable: $0 < \beta < 1$, and both tests compare rational numbers. Retain radius ρ around the queried set T, freezing the outer boundary as in Algorithm 4.

For $0 \le s \le t$, let V_s consist of the vertices at distance at most $h(t-s)$ from T. If $z \in V_s$ and $s \ge 1$, then $B_h(z) \subseteq V_{s-1}$. Every vertex in these sets has distance at most $ht < \rho$ from T, so all the updates used by the one-sweep inequality occur in the interior. Lemma B.1 therefore applies to $D_z(s) = \Delta_z(s)$ and $N(z) = B_h(z)$. At the queried vertices, which form $V_t = T$, it gives

$$\mathbb{P}(\Delta_z(t) > \delta) \le 2^{-t}\delta^{-p} \le \alpha/r.$$

A union bound over at most r queried coordinates proves the width guarantee of Theorem 6.1.

Each radius-ρ neighborhood has at most $1 + d + \cdots + d^\rho \le (d+1)^{\rho+1}$ vertices. We may therefore use

$$b = r(d+1)^{\rho+1}, \qquad E = S^{bt} \tag{45}$$

as bounds on the retained vertices and enumerated digit assignments. All these choices depend only on d, h, r, δ, α.

B.4 Exact arithmetic

Suppose the effective endpoints have at most Λ fractional binary digits. An update takes maxima and minima, forms an integer linear combination, and divides by the power of two S. It adds at most $\log S$ fractional digits. If b' interior vertices are updated, each endpoint calculation makes $b't$ updates. The two calculations share digits but use their own values, so the precision in either is at most

$$\Lambda + b't \log S \le \Lambda + \log E. \tag{46}$$

All positions remain in $[0,1]$, so their integer numerators need at most one additional bit. Digits and tallies have at most $\log S$ and $\log E + 1$ bits, respectively. This proves the remaining claims of Theorem 6.1.

C Parameter reference

Table 1 collects the parameters of the adversary and its analysis. The last two columns distinguish where a parameter's role is introduced from where its value is chosen and explained. All choices are fixed before the first query. Appendix B gives explicit choices for the digit alphabet, number of sweeps, and neighborhood radius in the local enclosure theorem.

Table 1: Parameters and their roles. Section references lead to the explanations in the main text.

Symbol	Value	Role	Introduced	Choice
k	Integer, $k \geq 64$	Refine a queried free item when it has this many neighbors.	§3.2	§4
H	2^k	Number of vertices refined in a selected directed path.	§3.2	§4.3
B	$(k+1)^{2H+1}$	Bounds the number of other vertices comparable with one vertex in its in-node DAG.	§5.3	§5.3
w_*	2^{-2kB}	Calibrates the finite calculations: a coordinate of at least this width guarantees certification.	§5.3	§5.3
η	$1/(100k)$	Accept an answer whose certified probability is at least $1/2 - \eta$.	§3.2	§4.1
$Q(n)$	$n \lceil \log n \rceil$	Number of answers before switching to a fixed consistent order.	§3.2	§3.2

References

[1] Mikhail J. Atallah and S. Rao Kosaraju. *An adversary-based lower bound for sorting.* Information Processing Letters 13(2), 55–57, 1981.

[2] Allan Borodin, Leonidas J. Guibas, Nancy A. Lynch, and Andrew C. Yao. *Efficient searching using partial ordering.* Information Processing Letters 12(2), 71–75, 1981. doi:10.1016/0020-0190(81)90005-3.

[3] Dana Richards and Pravin M. Vaidya. *On the distribution of comparisons in sorting algorithms.* BIT 28, 764–774, 1988. Authors' paper.

[4] Gerth Stølting Brodal, Shiva Chaudhuri, and Jaikumar Radhakrishnan. *The randomized complexity of maintaining the minimum.* Nordic Journal of Computing 3(4), 337–351, 1996. Authors' report, BRICS RS-96-40.

[5] Haim Kaplan, Or Zamir, and Uri Zwick. *The amortized cost of finding the minimum.* Proceedings of the Twenty-Sixth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 757–768, 2015. Published online 22 December 2014. doi:10.1137/1.9781611973730.51. Authors' paper.

[6] Haim Kaplan, Or Zamir, and Uri Zwick. *A sort of an adversary.* Proceedings of the Thirtieth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 1291–1310, 2019. doi:10.1137/1.9781611975482.79.

[7] Richard P. Stanley. *Two poset polytopes.* Discrete & Computational Geometry 1, 9–23, 1986. doi:10.1007/BF02187680. Author's copy.

[8] Graham Brightwell and Peter Winkler. *Counting linear extensions.* Order 8(3), 225–242, 1991. doi:10.1007/BF00383444.

[9] Martin Dyer, Alan Frieze, and Ravi Kannan. *A random polynomial-time algorithm for approximating the volume of convex bodies.* Journal of the ACM 38(1), 1–17, 1991. doi:10.1145/102782.102783. Authors' manuscript.

[10] Jeff Kahn and Jeong Han Kim. *Entropy and sorting.* Journal of Computer and System Sciences 51(3), 390–399, 1995. doi:10.1006/jcss.1995.1077.

[11] András Prékopa. *On logarithmic concave measures and functions.* Acta Scientiarum Mathematicarum 34, 335–343, 1973. Publisher's paper.